

数学建模 多元分析 10.1 10.2 题报告

2351114 大数据 朱俊泽

10.1 题目

10.1 表 10.1 是 1999 年中国省、自治区的城市规模结构特征的一些数据,试通过聚类分析将这些省、自治区进行分类。

表 10.1 城市规模结构特征数据

省、自治区	城市规模 /万人	城市首位度	城市指数	基尼系数	城市规模中位 值/万人
京津冀	699.70	1.4371	0.9364	0.7804	10.880
山西	179.46	1.8982	1.0006	0.5870	11.780
内蒙古	111.13	1.4180	0.6772	0.5158	17.775
辽宁	389.60	1.9182	0.8541	0.5762	26.320
吉林	211.34	1.7880	1.0798	0.4569	19.705
黑龙江	259.00	2.3059	0.3417	0.5076	23.480
苏沪	923.19	3.7350	2.0572	0.6208	22.160
浙江	139.29	1.8712	0.8858	0.4536	12.670
安徽	102.78	1.2333	0.5326	0.3798	27.375
福建	108.50	1.7291	0.9325	0.4687	11.120
江西	129.20	3.2454	1.1935	0.4519	17.080
山东	173.35	1.0018	0.4296	0.4503	21.215
河南	151.54	1.4927	0.6775	0.4738	13.940
湖北	434.46	7.1328	2.4413	0.5282	19.190
湖南	139.29	2.3501	0.8360	0.4890	14.250
广东	336.54	3.5407	1.3863	0.4020	22.195
广西	96.12	1.2288	0.6382	0.5000	14.340
海南	45.43	2.1915	0.8648	0.4136	8.730
川渝	365.01	1.6801	1.1486	0.5720	18.615
云南	146.00	6.6333	2.3785	0.5359	12.250
贵州	136.22	2.8279	1.2918	0.5984	10.470
西藏	11.79	4.1514	1.1798	0.6118	7.315
陕西	244.04	5.1194	1.9682	0.6287	17.800
甘肃	145.49	4.7515	1.9366	0.5806	11.650
青海	61.36	8.2695	0.8598	0.8098	7.420
宁夏	47.60	1.5078	0.9587	0.4843	9.730
新疆	128.67	3.8535	1.6216	0.4901	14.470

10.1.1 建模

本题目是一题多元分析题目，涉及对中国 27 个省、自治区城市规模结构特征数据进行聚类分析。具体步骤包括数据标准化、样本间距离计算、层次聚类以及聚类谱系图的绘制与解释。

原始数据以图片形式提供，包含 27 个省、自治区在城市规模、城市首位度、

城市指数、基尼 系数和城市规模中位值等五个指标上的数据。为了便于后续的计算和分析，我们首先将图片中 的表格数据手动提取并保存为 CSV 格式文件

CSV 文件结构如下：省、自治区城市规模/万人 城市首位度 城市指数 基尼系数城市规模中位值/万人。

	省、自治区	城市规模/万人	城市首位度	城市指数	基尼系数	城市规模中位值/万人
1	京津冀	699.70	1.4371	0.9364	0.7804	10.880
2	山西	179.46	1.8982	1.0006	0.5870	11.780
3	内蒙古	111.13	1.4180	0.6772	0.5158	17.775
4	辽宁	389.60	1.9182	0.8541	0.5762	26.320
5	吉林	211.34	1.7880	1.0798	0.4569	19.705
6	黑龙江	259.00	2.3059	0.3417	0.5076	23.480
7	苏沪	923.19	3.7350	2.0572	0.6208	22.160
8	浙江	139.29	1.8712	0.8858	0.4536	12.670
9	安徽	102.78	1.2333	0.5326	0.3798	27.375
10	福建	108.50	1.7291	0.9325	0.4687	11.120
11	江西	129.20	3.2454	1.1935	0.4519	17.080
12	山东	173.35	1.0018	0.4296	0.4503	21.215
13	河南	151.54	1.4927	0.6775	0.4738	13.940
14	湖北	434.46	7.1328	2.4413	0.5282	19.190
15	湖南	139.29	2.3501	0.8360	0.4890	14.250
16	广东	336.54	3.5407	1.3863	0.4020	22.195
17	广西	96.12	1.2288	0.6382	0.5000	14.340
18	海南	45.43	2.1915	0.8648	0.4136	8.730
19	川渝	365.01	1.6801	1.1486	0.5720	18.615
20	云南	146.00	6.6333	2.3785	0.5359	12.250
21	贵州	136.22	2.8279	1.2918	0.5984	10.470
22	西藏	11.79	4.1514	1.1798	0.6118	7.315
23	陕西	244.04	5.1194	1.9682	0.6287	17.800
24	甘肃	145.49	4.7515	1.9366	0.5806	11.650
25	青海	61.36	8.2695	0.8598	0.8098	7.420
26	宁夏	47.60	1.5078	0.9587	0.4843	9.730
27	新疆	128.67	3.8535	1.6216	0.4901	14.470

在多元分析中，不同指标的量纲和取值范围可能存在巨大差异，这会影响距离计算的准确性。 例如，城市规模的数值可能远大于城市首位度，导致城市规模在距离计算中占据主导地位。为 了消除这种量纲和数量级的影响，我们需要对数据进行标准化处理。采用以下公式对所有 a_{ij} 数据进行标准化：

$$b_{ij} = \frac{a_{ij} - \mu_j}{s_j}, i = 1, 2, \dots, 27; j = 1, 2, \dots, 5.$$

其中 $\mu_j = \frac{1}{27} \sum_{i=1}^{27} a_{ij}$, $s_j = \sqrt{\frac{1}{26} \sum_{i=1}^{27} (a_{ij} - \mu_j)^2}$ ($j = 1, 2, \dots, 5$)，其中 μ_j 和 s_j 也就是 j 指标的样本均值和样本方差， i 代表的就是每一个城市的数据值，总的就是他们。经过 $b_{ij} = \frac{a_{ij} - \mu_j}{s_j}$ 处理之后就实现了数据的正则化。同时，把 $b_{ij} = \frac{a_{ij} - \mu_j}{s_j}$ 中 b_{ij} 称作标准化的指标变量。

随后构建样本特征之间的距离，距离矩阵（ $d_{27 \times 27}$ ）利用欧几里得距离来展

示两个样本之间的距离，和相似度呈反比。其中 d_{ik} 计算方式为：

$$d_{ik} = \sqrt{\sum_{j=1}^3 (b_{ij} - b_{kj})^2}, i, k = 1, 2, \dots, 27.$$

其中 $b_{ij} = \frac{a_{ij} - \mu_j}{s_j}$ 为某一类数据的标准化特征。用最短距离来计算 27 个样本之间的最短距离，也就是：

$$D(G_p, G_q) = \min_{i \in G_p, k \in G_q} \{d_{ik}\}.$$

然后构造 27 个类，每一个类中质保函一个样本点，合并距离最近的类作为新的类，反复重复这个合并步骤知道类剩下一个，绘制聚类图。

10.1.2 代码

```
import pandas as pd
from scipy.spatial.distance import pdist, squareform
from scipy.cluster.hierarchy import linkage, dendrogram
import matplotlib.pyplot as plt
import numpy as np

# Load the data
df = pd.read_csv(r'D:\Mathematical Modeling\Mathematical-Modeling\大二下-数学建模课程\作业 6\data10_1.csv')

# Extract the province/region names and the numerical data
provinces = df.iloc[:, 0].tolist()
data = df.iloc[:, 1:].values

# (1) Data Standardization (Z-score standardization)
# The problem description mentions using  $(a_j - \mu_j) / s_j$ , where  $\mu_j$  and  $s_j$  are sample mean and sample standard deviation.
# This is exactly Z-score standardization.
# Calculate mean and standard deviation for each column (feature)
mu = np.mean(data, axis=0)
s = np.std(data, axis=0)
# Standardize the data
data_standardized = (data - mu) / s

# (2) Calculate Euclidean distance between 27 sample points
distance_matrix = pdist(data_standardized, metric='euclidean')
distance_square_matrix = squareform(distance_matrix)

# (3) & (4) Hierarchical Clustering using Single Linkage (shortest distance method)
```

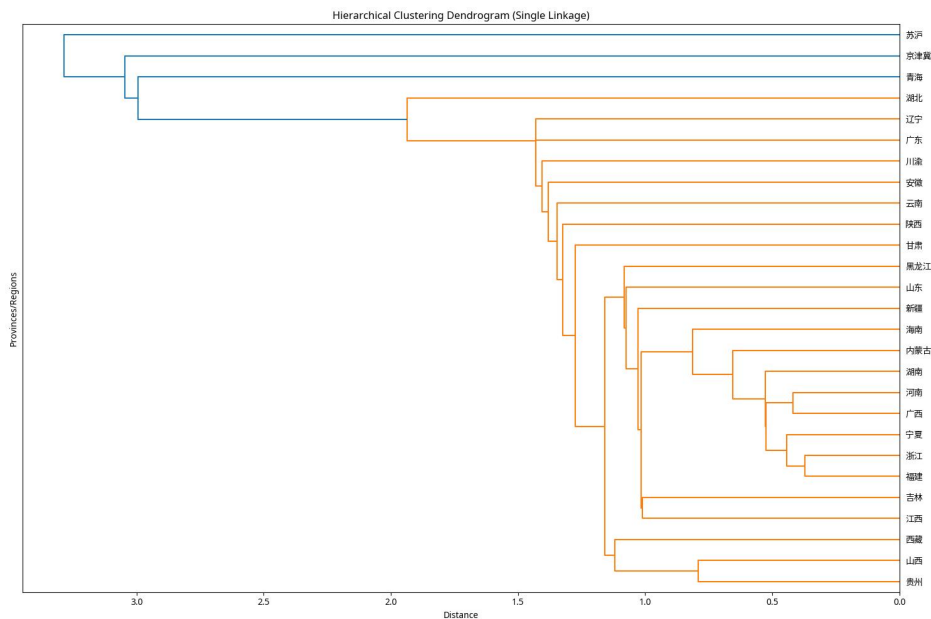
```

# The problem states to use the shortest distance method to measure distance between
classes.

# This corresponds to 'single' linkage in scipy's linkage function.
linked = linkage(data_standardized, method='single')
# (6) Plot the dendrogram
plt.figure(figsize=(15, 10))
dendrogram(linked,
            orientation='left',
            labels=provinces,
            distance_sort='descending',
            show_leaf_counts=True)
plt.title('Hierarchical Clustering Dendrogram (Single Linkage)')
plt.xlabel('Distance')
plt.ylabel('Provinces/Regions')
plt.tight_layout()
plt.savefig('dendrogram.png')
plt.show()

```

10.1.3 结果



本题目结果中看出，苏沪、京津冀、青海一类，其他省份一类。

10.2 题目

10.2 表 10.54 是我国 1984—2000 年宏观投资的一些数据,试利用主成分分析对投资效益进行分析和排序。

表 10.54 1984—2000 年宏观投资效益主要指标

年份	投资效果系数 (无时滞)	投资效果系数 (时滞一年)	全社会固定资产 交付使用率	建设项目 投产率	基建房屋 竣工率
1984	0.71	0.49	0.41	0.51	0.46
1985	0.40	0.49	0.44	0.57	0.50
1986	0.55	0.56	0.48	0.53	0.49
1987	0.62	0.93	0.38	0.53	0.47
1988	0.45	0.42	0.41	0.54	0.47
1989	0.36	0.37	0.46	0.54	0.48
1990	0.55	0.68	0.42	0.54	0.46
1991	0.62	0.29	0.38	0.56	0.46
1992	0.61	0.99	0.33	0.57	0.43
1993	0.71	0.93	0.35	0.66	0.44
1994	0.59	0.69	0.36	0.57	0.48
1995	0.41	0.47	0.40	0.54	0.48
1996	0.26	0.29	0.43	0.57	0.48
1997	0.14	0.16	0.43	0.55	0.47
1998	0.12	0.13	0.45	0.59	0.54
1999	0.22	0.25	0.44	0.58	0.52
2000	0.71	0.49	0.41	0.51	0.46

10.2.1 建模

本题目是一道多元分析题目，利用主成分分析法。主成分分析（Principal Component Analysis, PCA）是一种常用的数据降维技术，旨在通过线性变换将原始数据转换成一组新的、不相关的变量，这些新变量被称为主成分。主成分是原始变量的线性组合，并且能够尽可能多地保留原始数据的信息。在实际应用中，PCA 常用于数据预处理、特征提取、数据可视化和模式识别等领域。

在进行主成分分析之前，通常需要对原始数据进行标准化处理。这是因为不同指标可能具有不同的量纲和数量级，直接使用原始数据进行分析可能会导致量纲较大的指标在分析中占据主导地位，从而掩盖其他重要信息。标准化处理的目的是消除量纲影响，使所有指标在同一水平上进行比较。

data10_2.txt					
0.71	0.49	0.41	0.51	0.46	
0.40	0.49	0.44	0.57	0.50	
0.55	0.56	0.48	0.53	0.49	
0.62	0.93	0.38	0.53	0.47	
0.45	0.42	0.41	0.54	0.47	
0.36	0.37	0.46	0.54	0.48	
0.55	0.68	0.42	0.54	0.46	
0.62	0.90	0.38	0.56	0.46	
0.61	0.99	0.33	0.57	0.43	
0.71	0.93	0.35	0.66	0.44	
0.59	0.69	0.36	0.57	0.48	
0.41	0.47	0.40	0.54	0.48	
0.26	0.29	0.43	0.57	0.48	
0.14	0.16	0.43	0.55	0.47	
0.12	0.13	0.45	0.59	0.54	
0.22	0.25	0.44	0.58	0.52	
0.71	0.49	0.41	0.51	0.46	

原始数据被表示为一个矩阵 $A = (a_{ij})_{17 \times 5}$ ，其中 a_{ij} 表示第 i 年第 j 个指标变量 x_j 的取值。该数据矩阵来源于“表 10.2 1984—2000 年宏观投资效益主要指标”。

标准化处理采用 Z-score 标准化方法，进行 Z-score 标准化，使其均值为 0，标准差为 1。其公式如下：

$$\tilde{a}_{ij} = \frac{a_{ij} - \mu_j}{s_j}, \quad i = 1, 2, \dots, 17; j = 1, 2, \dots, 5$$

其中： a_{ij} 是原始数据矩阵 A 中第 i 行第 j 列的元素，即第 i 年第 j 个指标的取值 μ_j 是第 j 个指标的样本均值，计算公式为：

$$\mu_j = \frac{1}{17} \sum_{i=1}^{17} a_{ij}$$

s_j 是第 j 个指标的样本标准差，计算公式为：

$$s_j = \sqrt{\frac{1}{16} \sum_{i=1}^{17} (a_{ij} - \mu_j)^2}$$

标准化后的数据记为 $\tilde{a}_{ij}$ ，它们构成了标准化后的数据矩阵 $\tilde{A}$ 。

标准化数据后，需要计算标准化数据矩阵的相关系数矩阵 R 。相关系数矩阵描述了各个指标之间的线性关系强度和方向。在主成分分析中，通常基于相关系数矩阵进行计算，而不是协方差矩阵，尤其是在原始变量量纲不同的情况下，使用相关系数矩阵可以避免量纲对分析结果的影响。

相关系数矩阵 R 是一个 5×5 的对称矩阵，其元素 r_{jk} 表示第 j 个指标和第 k 个指标之间的相关系数。其计算公式为：

$$r_{jk} = \frac{\sum_{i=1}^{17} \tilde{a}_{ij} \tilde{a}_{ik}}{17-1}, \quad j, k = 1, 2, \dots, 5$$

其中：* $\tilde{a}_{ij}$ 是标准化后数据矩阵中第 i 行第 j 列的元素。* $\tilde{a}_{ik}$ 是标准化后数据矩阵中第 i 行第 k 列的元素。

计算出相关系数矩阵 R 后，下一步是计算其特征值（eigenvalues）和对应的特征向量（eigenvectors）。特征值和特征向量是主成分分析的核心，它们决定了主成分的构成和重要性。

对于一个 $p \times p$ 的相关系数矩阵 R （在本例中 $p = 5$ ），其特征值 λ 和特征向量 u 满足以下方程：

$$Ru = \lambda u$$

其中：* R 是相关系数矩阵。* u 是一个 $p \times 1$ 的列向量，表示特征向量。* λ 是一个标量，表示特征值。

通过求解上述方程，可以得到 p 个特征值 $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_p \geq 0$ 及其对应的 p 个单位特征向量 $u_1, u_2, \dots, u_p$ 。这些特征向量构成了新的坐标轴，即主成分。

在文档中，特征向量 u_j 被表示为 $u_j = [u_{1j}, u_{2j}, \dots, u_{5j}]^T$ 。每个特征向量的元素 u_{kj} 表示第 k 个原始标准化变量在第 j 个主成分上的载荷（或称系数）。

新的指标变量（主成分） y_j 是原始标准化变量 $\tilde{x}_k$ 的线性组合，其表达式为：

$$y_j = u_{1j}\tilde{x}_1 + u_{2j}\tilde{x}_2 + \dots + u_{5j}\tilde{x}_5$$

或者更一般地：

$$y_j = \sum_{k=1}^5 u_{kj} \tilde{x}_k$$

在计算出所有特征值和特征向量后，需要根据特征值的大小来评估每个主成分的重要性，并选择能够代表原始数据大部分信息的前 p 个主成分。衡量主成分重要性的指标是其**信息贡献率**和**累计贡献率**。

第 j 个主成分的信息贡献率 b_j 的计算公式为：

$$b_j = \frac{\lambda_j}{\sum_{k=1}^5 \lambda_k}, \quad j = 1, 2, \dots, 5$$

其中： * λ_j 是第 j 个特征值。 * $\sum_{k=1}^5 \lambda_k$ 是所有特征值的和。

信息贡献率表示第 j 个主成分解释原始数据总方差的比例。累计贡献率 α_p 是前 p 个主成分的信息贡献率之和，其计算公式为：

$$\alpha_p = \frac{\sum_{k=1}^p \lambda_k}{\sum_{k=1}^5 \lambda_k}$$

主成分得分（Principal Component Scores）是每个样本在各个主成分上的取值，它反映了每个样本在主成分方向上的投影。主成分得分的计算是将标准化后的原始数据投影到选定的主成分方向上。

假设我们选择了前 p 个主成分，其对应的特征向量构成了载荷矩阵 $U_p = [u_1, u_2, \dots, u_p]$ 。那么，每个样本的主成分得分 df_i 可以通过将标准化后的数据 $\tilde{A}_i$ （第 i 个样本的标准化数据行向量）与载荷矩阵 U_p 相乘得到：

$$DF = \tilde{A}U_p$$

其中： * DF 是主成分得分矩阵，其维度为 $N \times p$ （ N 为样本数量，本例中 $N = 17$ ）。 * $\tilde{A}$ 是标准化后的数据矩阵。 * U_p 是由选定的 p 个主成分的特征向量组成的矩阵。

为了对每个样本进行综合评价，通常会将选定的主成分得分进行加权求和，得到一个综合得分。权重通常是每个主成分的信息贡献率。

综合得分 Z 的计算公式为：

$$Z = \sum_{j=1}^p b_j y_j$$

其中： * y_j 是第 j 个主成分的得分。 * b_j 是第 j 个主成分的信息贡献率（或其归一化后的值）。 * p 是选定的主成分数量。

最后进行排序。

10.2.2 代码

```
import numpy as np
from scipy.stats import zscore
from sklearn.decomposition import PCA
import pandas as pd

# 1. 读取原始数据
# 假设 data10_2.txt 是一个包含原始数据的文本文件，例如 CSV 或 TSV 格式
# 这里使用 numpy.loadtxt，如果文件是其他格式，可能需要使用 pandas.read_csv
try:
    a = np.loadtxt("data10_2.txt")
except FileNotFoundError:
    print("错误：data10_2.txt 文件未找到。请确保文件存在于当前目录下。")
    # 为了演示，如果文件不存在，创建一个示例数据
    a = np.array([
        [0.71, 0.49, 0.41, 0.51, 0.46],
        [0.40, 0.49, 0.44, 0.57, 0.50],
        [0.55, 0.56, 0.48, 0.53, 0.49],
        [0.62, 0.93, 0.38, 0.53, 0.47],
        [0.45, 0.42, 0.41, 0.54, 0.47],
        [0.36, 0.37, 0.46, 0.54, 0.48],
```

```

        [0.55, 0.68, 0.42, 0.54, 0.46],
        [0.62, 0.90, 0.38, 0.56, 0.46],
        [0.61, 0.99, 0.33, 0.57, 0.43],
        [0.71, 0.93, 0.35, 0.66, 0.44],
        [0.59, 0.69, 0.36, 0.57, 0.48],
        [0.41, 0.47, 0.40, 0.54, 0.48],
        [0.26, 0.29, 0.43, 0.57, 0.48],
        [0.14, 0.16, 0.43, 0.55, 0.47],
        [0.12, 0.13, 0.45, 0.59, 0.54],
        [0.22, 0.25, 0.44, 0.58, 0.52],
        [0.71, 0.49, 0.41, 0.51, 0.46]
    ])

    print("使用示例数据进行演示。")

# 2. 数据标准化 (Z-score)
# scipy.stats.zscore 默认对列进行标准化
b = zscore(a)

# 3. 计算相关系数矩阵
r = np.corrcoef(b, rowvar=False) # rowvar=False 表示列代表变量

# 4. 主成分分析 (PCA)
# Matlab 的 pcacov(R) 返回特征向量、特征值和解释方差比例
# sklearn.decomposition.PCA 拟合后可以获取这些信息
# 注意: sklearn 的 PCA 默认是基于样本的协方差矩阵, 但如果输入是相关系数矩阵,
# 并且数据已经标准化, 效果是等价的。
# 这里我们直接对标准化后的数据 b 进行 PCA, 这等同于对相关系数矩阵进行 PCA
# 因为标准化后的数据的协方差矩阵就是相关系数矩阵。

pca = PCA()
pca.fit(b)

x = pca.components_.T # 特征向量 (Matlab 的 x), 需要转置使其列为特征向量
y = pca.explained_variance_ # 特征值 (Matlab 的 y)
z = pca.explained_variance_ratio_ * 100 # 解释方差比例 (Matlab 的 z), 转换为百分比
print("\n 特征向量 (x):\n", x)
print("\n 特征值 (y):\n", y)
print("\n 解释方差比例 (z):\n", z)

# 5. 修正特征向量的正负号
# Matlab 的 sign(sum(x)) 对应 numpy.sign(numpy.sum(x, axis=0))
f = np.sign(np.sum(x, axis=0))
x = x * f # 逐元素相乘, 修正特征向量
print("\n 修正后的特征向量 (x):\n", x)

# 6. 选取主成分的个数
num = 3

# 7. 计算各个主成分的得分
# df = b * x[:,1:num] 对应 numpy 的矩阵乘法和切片
df = np.dot(b, x[:, :num])
print("\n 主成分得分 (df):\n", df)

```

```

# 8. 计算综合得分
# tf = df * z(1:num)/100 对应 numpy 的矩阵乘法和逐元素除法
# 注意: z 已经是百分比, 所以这里不需要再除以 100, 除非 Matlab 的 z 是原始方差
# 根据文档描述, z 是解释方差比例, 所以这里直接使用
# 如果 Matlab 的 z 是原始方差, 则需要除以总方差
# 假设 Matlab 的 z 是解释方差比例的百分比, 所以这里直接用 z[:num] / 100
tf = np.dot(df, z[:num] / 100)
print("\n 综合得分 (tf):\n", tf)

# 9. 把得分按照从高到低的次序排列
# [stf,ind]=sort(tf,'descend\') 对应 numpy.argsort
ind = np.argsort(tf)[::-1] # 获取降序排列的索引
stf = tf[ind] # 根据索引获取排序后的得分
# Matlab 的 stf=stf\', ind=ind\' 只是转置, Python 中根据需要决定是否转置
# 这里保持为一维数组, 如果需要列向量形式, 可以 reshape
stf = stf.reshape(-1, 1)
# 修正年份数据和索引, 使其与原始数据行数匹配
years = np.arange(1984, 2001) # 从 1984 到 2000, 共 17 年
ind_0based = ind # ind 已经是 0-based 索引
print("\n 排序后的综合得分 (stf):\n", stf)
print("\n 排序索引 (ind):\n", ind_0based)
print("\n 排名和综合评价结果: ")
# 创建一个 DataFrame 以便更好地展示结果
results_df = pd.DataFrame({'年份': years[ind_0based.flatten()], '综合评价':
stf.flatten(), '排名': np.arange(1, len(stf) + 1)})
print(results_df.to_string(index=False))

```

10.2.3 结果

最后一步是对计算得到的综合得分进行排序, 以便对样本进行排名和比较。通常按照综合得分从高到低进行排序。

主成分分析 (PCA) 是一种强大的多元统计方法, 它通过正交变换将一组可能相关的变量转换为一组线性不相关的变量, 即主成分。这些主成分能够最大程度地保留原始数据的信息, 同时实现数据降维的目的。

代码结果如图所示:

特征向量 (x):

```
[[-0.49054217 -0.29344022 -0.51089699  0.18963273  0.61342066]
 [-0.52535146  0.04898756 -0.4336599  -0.12174796 -0.72022398]
 [ 0.48705734 -0.28119552 -0.37135078  0.68875986 -0.2672315 ]
 [-0.06705441  0.89811703 -0.14765822  0.38631158  0.13360356]
 [ 0.49158221  0.16064847 -0.62547502 -0.57060501  0.125419  ]]
```

特征值 (y):

```
[3.33022395 1.24137137 0.37206687 0.23992282 0.12891499]
```

解释方差比例 (z):

```
[62.6865684 23.36699053 7.00361169 4.51619427 2.42663511]
```

修正后的特征向量 (x):

```
[[ 0.49054217 -0.29344022  0.51089699  0.18963273 -0.61342066]
 [ 0.52535146  0.04898756  0.4336599  -0.12174796  0.72022398]
 [-0.48705734 -0.28119552  0.37135078  0.68875986  0.2672315 ]
 [ 0.06705441  0.89811703  0.14765822  0.38631158 -0.13360356]
 [-0.49158221  0.16064847  0.62547502 -0.57060501 -0.125419  ]]
```

排名和综合评价结果:

年份	综合评价	排名
1993	2.521710	1
1992	2.037682	2
1991	1.146537	3
1994	0.886857	4
1987	0.871587	5
1990	0.232740	6
2000	0.054734	7
1984	0.054734	8
1995	-0.261209	9
1988	-0.274392	10
1985	-0.545441	11
1996	-0.763280	12
1986	-0.802838	13
1989	-1.001392	14
1997	-1.182933	15
1999	-1.238442	16
1998	-1.736652	17